SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0304	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	P. Joshna	**Reg. No.:**	192472388
Section / Batch:	Slot B/2024	**Date:**	05/10/2026

Code of Conduct

I P.Joshna 192472388 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: P. Joshna

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1.Problem Statement:
A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies. **Develop a Python application that validates:**
Email Address
Password Mobile Number
using Regular
Expressions.
Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!) **Mobile Number**
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output
Display:
Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Objective: - To develop a Python program using regular expressions to validate email addresses, passwords, and mobile numbers according to predefined validation rules. **Program Code: -**

```
import re email = input("Enter Email: ") password = input("Enter Password: ") mobile =
input("Enter Mobile Number: ") email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-
z]+\.(com|org|edu|net|in)$" password_pattern = r"^(?=.*[A-Z])(?=.*[a-
z])(?=.*\d)(?=.*[@#$%&!]).{8,}$" mobile_pattern = r"^[6-9]\d{9}$" if re.match(email_pattern,
email):
   print("Valid Email") else:
   print("Invalid Email") if
re.match(password_pattern, password):


   print("Strong Password")
else:
```

```
    print("Weak Password") if
re.match(mobile_pattern, mobile):
print("Valid Mobile Number") else:
    print("Invalid Mobile Number")
```

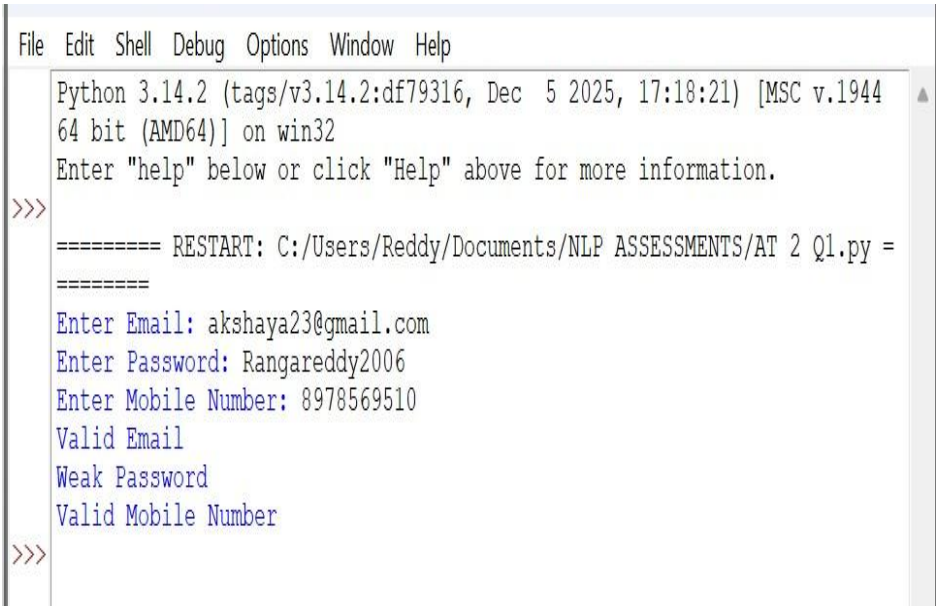
Sample Input: -

Enter Email: akshya123@gmail.com

Enter Password: Akshya@123

Enter Mobile Number: 9876543210

Output: -



Result: - Thus, the Regular Expression-based Email, Password, and Mobile Number Validator was successfully implemented and executed.

Section 2: Design a Finite State Automata (FSA) Simulator

2.Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description States
  - Input alphabet
  - Transition table
  - Initial state Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

q0 → q1 → q0 → q1 → q2

Accepted

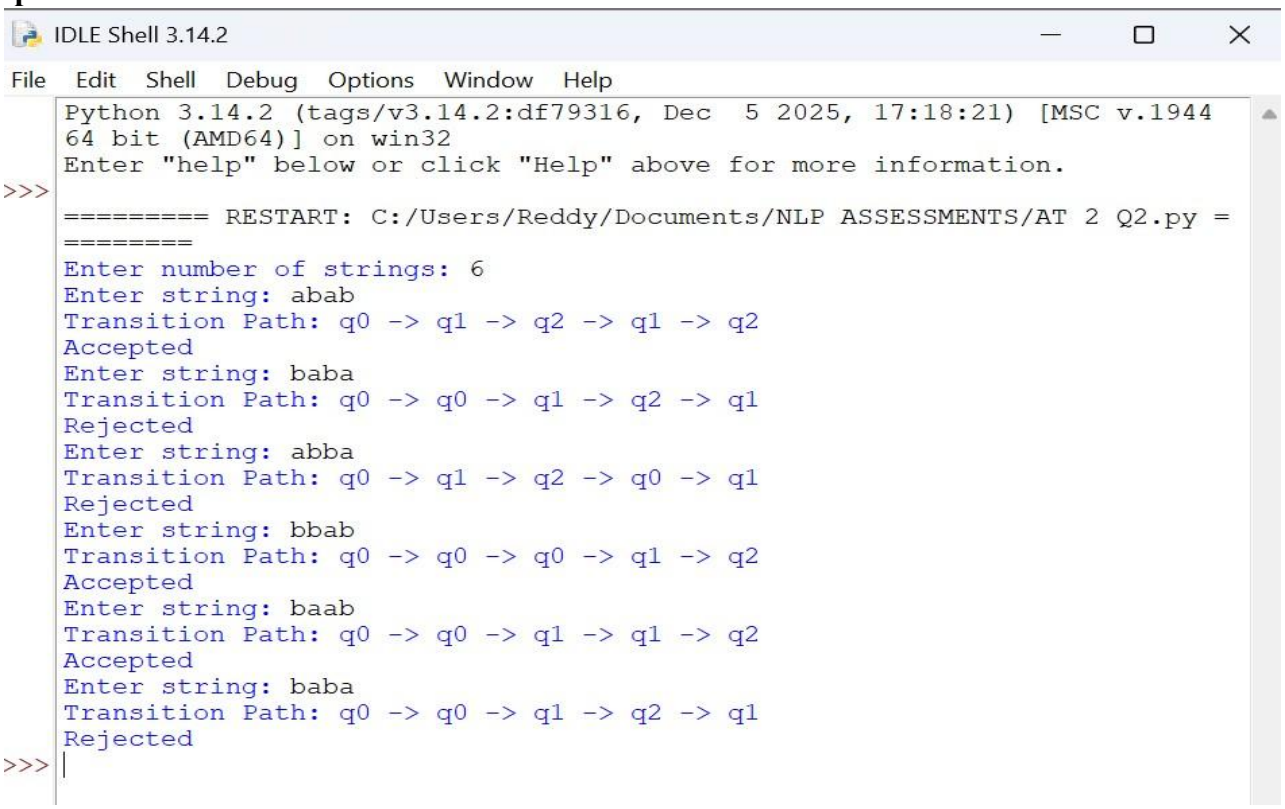
Objective: -

To develop a Python-based Deterministic Finite Automata (DFA) simulator that accepts a DFA description, processes multiple input strings, displays the transition path, and determines whether each string is Accepted or Rejected.

Program Code: -

```
states = ["q0", "q1", "q2"]
alphabet = ["a", "b"] transition
= {
   "q0": {"a": "q1", "b": "q0"},
   "q1": {"a": "q1", "b": "q2"},
   "q2": {"a": "q1", "b": "q0"}
} initial_state = "q0" final_states = ["q2"]
n = int(input("Enter number of strings: "))
for i in range(n):     string = input("Enter
string: ")     current = initial_state     path
= [current]     for symbol in string:
current = transition[current][symbol]
path.append(current)
   print("Transition Path:", " -> ".join(path))
if current in final_states:
      print("Accepted")
else:
      print("Rejected")
```

Output: -



Result: -The Python program successfully implemented a Deterministic Finite Automata (DFA) simulator to process multiple input strings and display their transition paths. The program correctly identified each string as Accepted or Rejected based on the final state.

Section 3: Smart Pattern Matching Engine using Regular Expressions

1.Problem Statement
Develop a Python-based text search engine that performs pattern matching using Regular Expressions. **The system should support:**
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search
Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
Expected Output
Display all matching patterns based on user selection.

Objective: -To develop a Python-based text search engine using Regular Expressions to perform date, phone number, hashtag, mention, prefix, and suffix pattern matching.

Program: - import
re
text = input("Enter Text: ") while
True:
print("\n1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix") print("6.
Search Suffix")
print("7. Exit") choice = input("Enter your
choice: ") if choice == "1": result =
re.findall(r"\b\d{2}/\d{2}/\d{4}\b", text)
print("Dates Found:", result) elif choice == "2":
result = re.findall(r"\b[6-9]\d{9}\b", text)
print("Phone Numbers Found:", result) elif
choice == "3": result =

```
re.findall(r"#\w+", text)        print("Hashtags
Found:", result)     elif choice == "4":
        result = re.findall(r"@\w+", text)
print("Mentions Found:", result)
elif choice == "5":         prefix =
input("Enter Prefix: ")         result =
re.findall(r"\b" + re.escape(prefix) +
r"\w*", text, re.IGNORECASE)
print("Prefix Matches:", result)     elif
choice == "6":         suffix =
input("Enter Suffix: ")
        result = re.findall(r"\b\w*" + re.escape(suffix) + r"\b", text, re.IGNORECASE)
print("Suffix Matches:", result)     elif choice == "7":         print("Exiting...")
break     else:
        print("Invalid Choice")
```

Output:

```
===== RESTART: C:/Users/Reddy/Documents/NLP ASSESSMENTS/AT 2 Q2.py =====
Enter Text: Meeting on 12/09/2026 Call 9876543210 #NLP @OpenAI natural l
anguage processing

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 1
Dates Found: ['12/09/2026']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 2
Phone Numbers Found: ['9876543210']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 4
Mentions Found: ['@OpenAI']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
```

Result: -The Python program successfully performed pattern matching using Regular Expressions for dates, phone numbers, hashtags, mentions, prefixes, and suffixes.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____